

General Data for Model

Erika Nares

2026-05-31

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.5
## ✓ forcats    1.0.0      ✓ stringr    1.5.1
## ✓ ggplot2    4.0.0      ✓ tibble     3.2.1
## ✓ lubridate  1.9.3      ✓ tidyr      1.3.1
## ✓ purrr      1.2.0
## — Conflicts — tidyverse_conflicts() —
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(reticulate)
```

```
## Warning: package 'reticulate' was built under R version 4.4.3
```

1. Getting game log data

```
from nba_api.stats.endpoints import leaguegamelog

def get_game_logs(season_year):

    logs = leaguegamelog.LeagueGameLog(
        season = season_year,
        season_type_all_star = "Regular Season",
        player_or_team_abbreviation = "T",
        timeout = 60
    )

    df_logs = logs.get_data_frames()[0]

    return df_logs
```

2. Getting team data

```
from nba_api.stats.endpoints import leaguedashteamstats
```

```
def get_teams(season_year):
```

```
    teams = leaguedashteamstats.LeagueDashTeamStats(  
        season = season_year,  
        season_type_all_star = "Regular Season"  
    )
```

```
    df_teams = teams.get_data_frames()[0]
```

```
    return df_teams
```

3. Retrieve SOS (Strength of Schedule) to calculate SRS (Simple Rating System = Average Point Differential + SOS)

```

library(dplyr)
library(stringr)

get_sos <- function(season_year){

  df_logs <- py$get_game_logs(season_year)
  df_teams <- py$get_teams(season_year)

  # Make lookup table to find the full name from abbreviations
  abbrev_lookup <- df_logs %>%
    select(Team_Abbreviation, Team_Name) %>%
    distinct() %>%
    rename(
      opp_abbreviation = Team_Abbreviation,
      opp_team_name = Team_Name
    )

  # Get opponent abbreviation from matchup in df_logs
  df_logs <- df_logs %>%
    select(
      Team_Name,
      Team_Abbreviation,
      Matchup
    ) %>%
    mutate(
      opp_abbreviation = str_sub(Matchup, -3, -1)
    ) %>%
    left_join(
      abbrev_lookup,
      by = "opp_abbreviation"
    )

  # Get opponent win percentage table from df_teams
  opponent_win_pct <- df_teams %>%
    select(
      Team_Name,
      W_Pct
    ) %>%
    rename(
      opp_team_name = Team_Name,
      opp_w_pct = W_Pct
    )

  # Join opponent win percentage using opponent team name
  df_logs <- df_logs %>%
    left_join(
      opponent_win_pct,
      by = "opp_team_name"
    )

  # Average opponent win percentage for each team
  sos <- df_logs %>%
    group_by(
      Team_Name
    ) %>%
    summarize(

```

```

      SOS_REG = mean(OPP_W_PCT, na.rm = TRUE),
      .groups = "drop"
    )

    return(sos)
  }

head(get_sos("2015-16"))

```

```

## # A tibble: 6 × 2
##   TEAM_NAME      SOS_REG
##   <chr>         <dbl>
## 1 Atlanta Hawks    0.500
## 2 Boston Celtics   0.489
## 3 Brooklyn Nets    0.510
## 4 Charlotte Hornets 0.489
## 5 Chicago Bulls     0.500
## 6 Cleveland Cavaliers 0.483

```

4. This calculates how each team performed against each opponent in the regular season.

```

get_head_to_head_data <- function(season_year){

  df_logs <- py$get_game_logs(season_year)

  head_to_head <- df_logs %>%
    select(
      TEAM_NAME,
      TEAM_ABBREVIATION,
      MATCHUP,
      WL,
      PLUS_MINUS
    ) %>%
    mutate(
      OPP_ABBREVIATION = str_sub(MATCHUP, -3, -1),
      WIN = ifelse(WL == "W", 1, 0)
    )

  # Make lookup table to find the full name from abbreviations
  abbrev_lookup <- df_logs %>%
    select(TEAM_ABBREVIATION, TEAM_NAME) %>%
    distinct() %>%
    rename(
      OPP_ABBREVIATION = TEAM_ABBREVIATION,
      OPP_TEAM_NAME = TEAM_NAME
    )

  head_to_head <- head_to_head %>%
    left_join(
      abbrev_lookup,
      by = "OPP_ABBREVIATION"
    ) %>%
    group_by(
      TEAM_NAME,
      OPP_TEAM_NAME
    ) %>%
    summarize(
      HEAD_TO_HEAD_GAMES = n(),
      HEAD_TO_HEAD_WINS = sum(WIN),
      HEAD_TO_HEAD_WIN_PCT = mean(WIN),
      HEAD_TO_HEAD_POINT_DIFF = mean(PLUS_MINUS),
      .groups = "drop"
    ) %>%
    mutate(
      season = season_year
    )

  return(head_to_head)
}

```

```

head(get_head_to_head_data("2015-16"))

```

```
## # A tibble: 6 × 7
##   TEAM_NAME      OPP_TEAM_NAME      HEAD_TO_HEAD_GAMES HEAD_TO_HEAD_WINS
##   <chr>          <chr>                <int>             <dbl>
## 1 Atlanta Hawks Boston Celtics          4              3
## 2 Atlanta Hawks Brooklyn Nets          3              2
## 3 Atlanta Hawks Charlotte Hornets        4              3
## 4 Atlanta Hawks Chicago Bulls            4              4
## 5 Atlanta Hawks Cleveland Cavaliers       3              0
## 6 Atlanta Hawks Dallas Mavericks          2              2
## # i 3 more variables: HEAD_TO_HEAD_WIN_PCT <dbl>,
## #   HEAD_TO_HEAD_POINT_DIFF <dbl>, season <chr>
```

5. This creates the main team-level dataset for one season and merges them with the playoff win percentage.

```

import time
import pandas as pd
from nba_api.stats.endpoints import leaguedashteamstats

def get_season_data(season_year):

    reg_adv = leaguedashteamstats.LeagueDashTeamStats(
        season = season_year,
        season_type_all_star = "Regular Season",
        measure_type_detailed_defense = "Advanced"
    )

    df_reg_adv = reg_adv.get_data_frames()[0]

    df_reg_adv = df_reg_adv[[
        "TEAM_ID",
        "TEAM_NAME",
        "W_PCT",
        "OFF_RATING",
        "DEF_RATING",
        "NET_RATING",
        "TS_PCT",
        "EFG_PCT",
        "OREB_PCT",
        "TM_TOV_PCT"
    ]]

    df_reg_adv = df_reg_adv.rename(columns = {
        "W_PCT": "W_PCT_REG",
        "OFF_RATING": "OFF_RATING_REG",
        "DEF_RATING": "DEF_RATING_REG",
        "NET_RATING": "NET_RATING_REG",
        "TS_PCT": "TS_PCT_REG",
        "EFG_PCT": "EFG_PCT_REG",
        "OREB_PCT": "OREB_PCT_REG",
        "TM_TOV_PCT": "TOV_PCT_REG"
    })

    df_reg_adv["PLUS_MINUS_REG"] = (
        df_reg_adv["OFF_RATING_REG"] - df_reg_adv["DEF_RATING_REG"]
    )

    time.sleep(1)

    reg_traditional = leaguedashteamstats.LeagueDashTeamStats(
        season = season_year,
        season_type_all_star = "Regular Season"
    )

    df_reg_traditional = reg_traditional.get_data_frames()[0]

    df_reg_traditional["THREE_PT_ATTEMPT_RATE_REG"] = (
        df_reg_traditional["FG3A"] / df_reg_traditional["FGA"]
    )

    df_reg_traditional = df_reg_traditional[[

```

```

        "TEAM_ID",
        "THREE_PT_ATTEMPT_RATE_REG"
    ]]

    time.sleep(1)

    po_traditional = leaguedashteamstats.LeagueDashTeamStats(
        season = season_year,
        season_type_all_star = "Playoffs"
    )

    df_po_traditional = po_traditional.get_data_frames()[0]

    df_po_traditional = df_po_traditional[[
        "TEAM_ID",
        "W_PCT"
    ]]

    df_po_traditional = df_po_traditional.rename(columns = {
        "W_PCT": "W_PCT_P0"
    })

    df = pd.merge(df_reg_adv, df_reg_traditional, on = "TEAM_ID")
    df = pd.merge(df, df_po_traditional, on = "TEAM_ID")

    df["season"] = season_year

    return df
get_season_data("2015-16").head()

```

```

##      TEAM_ID      TEAM_NAME  ...  W_PCT_P0  season
## 0  1610612737    Atlanta Hawks  ...    0.400  2015-16
## 1  1610612738    Boston Celtics  ...    0.333  2015-16
## 2  1610612766  Charlotte Hornets  ...    0.429  2015-16
## 3  1610612739  Cleveland Cavaliers  ...    0.762  2015-16
## 4  1610612742    Dallas Mavericks  ...    0.200  2015-16
##
## [5 rows x 14 columns]

```

6. Applies `get_season_data` to past 10 seasons.

```

years = [
    "2015-16", "2016-17", "2017-18", "2018-19", "2019-20",
    "2020-21", "2021-22", "2022-23", "2023-24", "2024-25"
]

all_season_data = []

for year in years:
    season_df = get_season_data(year)
    all_season_data.append(season_df)

combined_df = pd.concat(all_season_data, ignore_index = True)

```



```
head(py$combined_df)
```

```
##      TEAM_ID      TEAM_NAME W_PCT_REG OFF_RATING_REG DEF_RATING_REG
## 1 1610612737    Atlanta Hawks    0.585         104.6         100.8
## 2 1610612738    Boston Celtics    0.585         105.8         102.5
## 3 1610612766  Charlotte Hornets    0.585         106.4         103.7
## 4 1610612739 Cleveland Cavaliers    0.695         110.2         103.9
## 5 1610612742    Dallas Mavericks    0.512         105.5         105.9
## 6 1610612765    Detroit Pistons    0.537         105.3         104.9
##  NET_RATING_REG TS_PCT_REG EFG_PCT_REG OREB_PCT_REG TOV_PCT_REG PLUS_MINUS_REG
## 1           3.9      0.552      0.516      0.234      0.152           3.8
## 2           3.3      0.531      0.488      0.290      0.138           3.3
## 3           2.7      0.545      0.502      0.244      0.129           2.7
## 4           6.3      0.558      0.524      0.294      0.143           6.3
## 5          -0.4      0.544      0.502      0.241      0.132          -0.4
## 6           0.3      0.522      0.491      0.313      0.140           0.4
##  THREE_PT_ATTEMPT_RATE_REG W_PCT_P0  season
## 1           0.3359815      0.400 2015-16
## 2           0.2927029      0.333 2015-16
## 3           0.3481653      0.429 2015-16
## 4           0.3524971      0.762 2015-16
## 5           0.3394203      0.200 2015-16
## 6           0.3030902      0.000 2015-16
```

7. Joins SOS data and creates SRS_REG to join with the main df.

```
combined_df <- py$combined_df

years <- c(
  "2015-16", "2016-17", "2017-18", "2018-19", "2019-20",
  "2020-21", "2021-22", "2022-23", "2023-24", "2024-25"
)

all_sos <- map_dfr(years, function(year) {
  Sys.sleep(5)
  get_sos(year) %>%
    mutate(season = year)
})

combined_df <- combined_df %>%
  left_join(
    all_sos,
    by = c("TEAM_NAME", "season")
  ) %>%
  mutate(
    SRS_REG = PLUS_MINUS_REG + SOS_REG
  )

head(combined_df)
```

##	TEAM_ID	TEAM_NAME	W_PCT_REG	OFF_RATING_REG	DEF_RATING_REG
## 1	1610612737	Atlanta Hawks	0.585	104.6	100.8
## 2	1610612738	Boston Celtics	0.585	105.8	102.5
## 3	1610612766	Charlotte Hornets	0.585	106.4	103.7
## 4	1610612739	Cleveland Cavaliers	0.695	110.2	103.9
## 5	1610612742	Dallas Mavericks	0.512	105.5	105.9
## 6	1610612765	Detroit Pistons	0.537	105.3	104.9

##	NET_RATING_REG	TS_PCT_REG	EFG_PCT_REG	OREB_PCT_REG	TOV_PCT_REG	PLUS_MINUS_REG
## 1	3.9	0.552	0.516	0.234	0.152	3.8
## 2	3.3	0.531	0.488	0.290	0.138	3.3
## 3	2.7	0.545	0.502	0.244	0.129	2.7
## 4	6.3	0.558	0.524	0.294	0.143	6.3
## 5	-0.4	0.544	0.502	0.241	0.132	-0.4
## 6	0.3	0.522	0.491	0.313	0.140	0.4

##	THREE_PT_ATTEMPT_RATE_REG	W_PCT_P0	season	SOS_REG	SRS_REG
## 1	0.3359815	0.400	2015-16	0.5000610	4.3000610
## 2	0.2927029	0.333	2015-16	0.4890610	3.7890610
## 3	0.3481653	0.429	2015-16	0.4893415	3.1893415
## 4	0.3524971	0.762	2015-16	0.4829512	6.7829512
## 5	0.3394203	0.200	2015-16	0.5073415	0.1073415
## 6	0.3030902	0.000	2015-16	0.4949634	0.8949634

8. Correlation per season with postseason win percentage

```

cors <- combined_df %>%
  group_by(season) %>%
  summarize(
    win_pct_cor = cor(W_PCT_REG, W_PCT_P0),
    net_rating_cor = cor(NET_RATING_REG, W_PCT_P0),
    plus_minus_cor = cor(PLUS_MINUS_REG, W_PCT_P0),
    off_rating_cor = cor(OFF_RATING_REG, W_PCT_P0),
    def_rating_cor = cor(DEF_RATING_REG, W_PCT_P0),
    ts_pct_cor = cor(TS_PCT_REG, W_PCT_P0),
    efg_pct_cor = cor(EFG_PCT_REG, W_PCT_P0),
    three_pt_attempt_rate_cor = cor(THREE_PT_ATTEMPT_RATE_REG, W_PCT_P0),
    tov_pct_cor = cor(TOV_PCT_REG, W_PCT_P0),
    oreb_pct_cor = cor(OREB_PCT_REG, W_PCT_P0),
    sos_cor = cor(SOS_REG, W_PCT_P0),
    srs_cor = cor(SRS_REG, W_PCT_P0),
    .groups = "drop"
  )

cors

```

```
## # A tibble: 10 × 13
##   season win_pct_cor net_rating_cor plus_minus_cor off_rating_cor
##   <chr>      <dbl>      <dbl>      <dbl>      <dbl>
## 1 2015-16    0.709      0.781      0.780      0.669
## 2 2016-17    0.795      0.771      0.769      0.682
## 3 2017-18    0.583      0.382      0.384      0.383
## 4 2018-19    0.810      0.665      0.666      0.844
## 5 2019-20    0.684      0.639      0.633      0.413
## 6 2020-21    0.768      0.782      0.780      0.709
## 7 2021-22    0.625      0.666      0.666     -0.130
## 8 2022-23    0.216      0.211      0.210      0.384
## 9 2023-24    0.695      0.626      0.625      0.456
## 10 2024-25    0.686      0.608      0.610      0.536
## # i 8 more variables: def_rating_cor <dbl>, ts_pct_cor <dbl>,
## #   efg_pct_cor <dbl>, three_pt_attempt_rate_cor <dbl>, tov_pct_cor <dbl>,
## #   oreb_pct_cor <dbl>, sos_cor <dbl>, srs_cor <dbl>
```

9. Average correaltion over past ten seasons

```
avg_cors <- cors %>%
  summarize(across(everything(), mean, na.rm = TRUE))
```

```
## Warning: There were 2 warnings in `summarize()`.
## The first warning was:
## i In argument: `across(everything(), mean, na.rm = TRUE)`.
## Caused by warning:
## ! The `...` argument of `across()` is deprecated as of dplyr 1.1.0.
## Supply arguments directly to `.fns` through an anonymous function instead.
##
## # Previously
##   across(a:b, mean, na.rm = TRUE)
##
## # Now
##   across(a:b, \(x) mean(x, na.rm = TRUE))
## i Run `dplyr::last_dplyr_warnings()` to see the 1 remaining warning.
```

```
avg_cors
```

```
## # A tibble: 1 × 13
##   season win_pct_cor net_rating_cor plus_minus_cor off_rating_cor def_rating_cor
##   <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
## 1    NA    0.657      0.613      0.612      0.495     -0.234
## # i 7 more variables: ts_pct_cor <dbl>, efg_pct_cor <dbl>,
## #   three_pt_attempt_rate_cor <dbl>, tov_pct_cor <dbl>, oreb_pct_cor <dbl>,
## #   sos_cor <dbl>, srs_cor <dbl>
```

10. Correlations of variables with each other through matrix and colored matrix

```
predictors <- combined_df %>%  
  select(  
    W_PCT_REG,  
    NET_RATING_REG,  
    PLUS_MINUS_REG,  
    OFF_RATING_REG,  
    DEF_RATING_REG,  
    TS_PCT_REG,  
    EFG_PCT_REG,  
    THREE_PT_ATTEMPT_RATE_REG,  
    TOV_PCT_REG,  
    OREB_PCT_REG,  
    SOS_REG,  
    SRS_REG  
  )
```

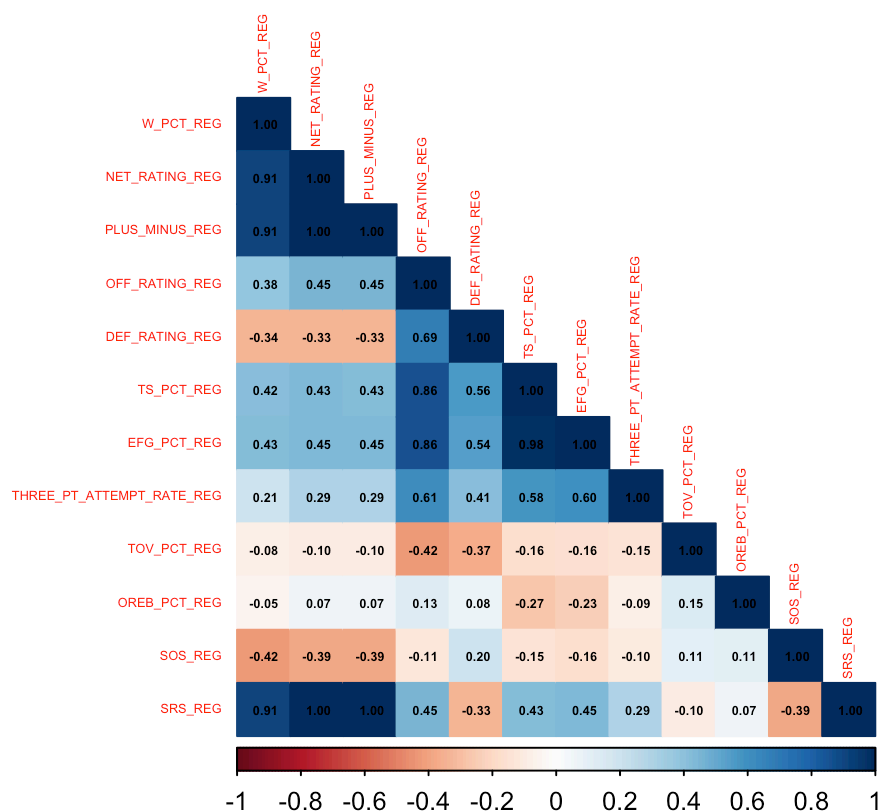
```
round(cor(predictors, use = "complete.obs"), 2)
```

##	W_PCT_REG	NET_RATING_REG	PLUS_MINUS_REG	
## W_PCT_REG	1.00	0.91	0.91	
## NET_RATING_REG	0.91	1.00	1.00	
## PLUS_MINUS_REG	0.91	1.00	1.00	
## OFF_RATING_REG	0.38	0.45	0.45	
## DEF_RATING_REG	-0.34	-0.33	-0.33	
## TS_PCT_REG	0.42	0.43	0.43	
## EFG_PCT_REG	0.43	0.45	0.45	
## THREE_PT_ATTEMPT_RATE_REG	0.21	0.29	0.29	
## TOV_PCT_REG	-0.08	-0.10	-0.10	
## OREB_PCT_REG	-0.05	0.07	0.07	
## SOS_REG	-0.42	-0.39	-0.39	
## SRS_REG	0.91	1.00	1.00	
##	OFF_RATING_REG	DEF_RATING_REG	TS_PCT_REG	EFG_PCT_REG
## W_PCT_REG	0.38	-0.34	0.42	0.43
## NET_RATING_REG	0.45	-0.33	0.43	0.45
## PLUS_MINUS_REG	0.45	-0.33	0.43	0.45
## OFF_RATING_REG	1.00	0.69	0.86	0.86
## DEF_RATING_REG	0.69	1.00	0.56	0.54
## TS_PCT_REG	0.86	0.56	1.00	0.98
## EFG_PCT_REG	0.86	0.54	0.98	1.00
## THREE_PT_ATTEMPT_RATE_REG	0.61	0.41	0.58	0.60
## TOV_PCT_REG	-0.42	-0.37	-0.16	-0.16
## OREB_PCT_REG	0.13	0.08	-0.27	-0.23
## SOS_REG	-0.11	0.20	-0.15	-0.16
## SRS_REG	0.45	-0.33	0.43	0.45
##	THREE_PT_ATTEMPT_RATE_REG	TOV_PCT_REG	OREB_PCT_REG	
## W_PCT_REG		0.21	-0.08	-0.05
## NET_RATING_REG		0.29	-0.10	0.07
## PLUS_MINUS_REG		0.29	-0.10	0.07
## OFF_RATING_REG		0.61	-0.42	0.13
## DEF_RATING_REG		0.41	-0.37	0.08
## TS_PCT_REG		0.58	-0.16	-0.27
## EFG_PCT_REG		0.60	-0.16	-0.23
## THREE_PT_ATTEMPT_RATE_REG		1.00	-0.15	-0.09
## TOV_PCT_REG		-0.15	1.00	0.15
## OREB_PCT_REG		-0.09	0.15	1.00
## SOS_REG		-0.10	0.11	0.11
## SRS_REG		0.29	-0.10	0.07
##	SOS_REG	SRS_REG		
## W_PCT_REG	-0.42	0.91		
## NET_RATING_REG	-0.39	1.00		
## PLUS_MINUS_REG	-0.39	1.00		
## OFF_RATING_REG	-0.11	0.45		
## DEF_RATING_REG	0.20	-0.33		
## TS_PCT_REG	-0.15	0.43		
## EFG_PCT_REG	-0.16	0.45		
## THREE_PT_ATTEMPT_RATE_REG	-0.10	0.29		
## TOV_PCT_REG	0.11	-0.10		
## OREB_PCT_REG	0.11	0.07		
## SOS_REG	1.00	-0.39		
## SRS_REG	-0.39	1.00		

```
library(corrplot)
```

```
## corrrplot 0.95 loaded
```

```
corrrplot(  
  cor(predictors, use = "complete.obs"),  
  method = "color",  
  type = "lower",  
  addCoef.col = "black",  
  number.cex = 0.4,  
  tl.cex = 0.4  
)
```



Based on the

correlation with win percentage and

1. NET_RATING_REG 2. TS_PCT_REG 3. THREE_PT_ATTEMPT_RATE_REG

TO DO:

1. Add more visuals
2. See how the difference in these variables correlates with the winner of each series

NEXT DOCS

1. Data for matchups only
 1. how each team performed during regular season against team they are playing against and see how that correlates with the winner of the series
 2. Will want to add EDA visuals
2. Modelling

1. Choose 1 or 2 types of models (maybe Logistic Regression and RF), finalize our predictors (our target variable will be whether team A or team B wins)
2. We can probably refer to PSTAT 131 Workflow and just copy that